Exercise 10: *sSDT* Protocol Checkers

Objective: *Introduction to protocol checkers as coroutines running in parallel with a simulation.*

Task: *Implementation and integration of protocol checkers for the sSDT interfaces of the Saturation Filter testbench.*

The files for this exercise are in `<ROOT>/lecture07/exercises/E10_ssdt_protocol_checkers/`. This directory contains the RTL and testbench files. In the following sections, file names are relative to `tb/`.

The *Saturation Filter* has an input and an output *sSDT* interface. In this exercise, you will implement a reusable checker and instantiate it for both interfaces.

Part 1: *sSDT* Protocol Checker

In `ssdt_interface_assertions.py` create a class named `ssdt_interface_assert_check`.

- The constructor shall accept and store a virtual *sSDT* interface:
  - `clk`
  - `rst`
  - `data`
  - `valid`
- The checker shall have a `passed` member variable, initially set to `True`. Set `passed` to `False` if any assertion fails.
- In the checker implement three assertion coroutines, one for each *sSDT* protocol requirement (PR) defined in `<ROOT>/lecture07/exercises/exercise_general_info.pdf`:
  - **PR1: Reset requirement**
  - **PR2: Data validity requirement**
  - **PR3: Data invalidity requirement**
- Each assertion shall sample the interface synchronously to `clk`.

The following example shows the structure of an assertion coroutine. Implement the PR1, PR2, and PR3 assertions using this structure.

```
async def example_assertion(self):
    while True:
        await RisingEdge(self.vif.clk)
        await ReadOnly()
        try:
            assert <condition>, "<failure message>"
        except AssertionError:
            self.passed = False
```

Each assertion coroutine must continue checking for the duration of the simulation. Add a useful message to each assertion so that a failed protocol requirement can be identified in the simulation output.

Part 2: Testbench Integration

Integrate the checker into `sat_tb_base_test.py`.

- Import `ssdt_interface_assert_check` from `ssdt_interface_assertions.py`.
- Add checker handles `producer_assert_check` and `consumer_assert_check` to the test class constructor.
- In `build_phase`, create a input-interface checker and a output-interface checker with the producer and consumer iterfaces respectively.
- In `run_phase`, start all assertion coroutines.
- In `report_phase`, assert the `passed` attribute of both checker instances so that a protocol violation fails the test.

Part 3: Testing the checkers

Verify that all configured tests complete without protocol-checker failures. Introduce a violation of each protocol requirement and verify that the corresponding checker records the failure and causes the test to fail in `report_phase`. Restore the testbench after each experiment.